7.5 Adaptive Control of a Real DC Motor

Example 7.6 of the reader cites the experiment by Abram and Fieg, where the adaptive controller is implemented on an Arduino Due [1, 2]. The present section realises the same self-tuning pole-placement idea on an own test rig. The hardware consists of an ESP32, an L298N H-bridge, and a DC motor with a quadrature encoder. The controller is not hand-coded on the microcontroller; instead, the real-time code is generated automatically from the Simulink model.

The structure follows the indirect adaptive-control scheme of Figure 7.1 in the reader. The plant parameters are estimated recursively, the controller is redesigned from these estimates, and the resulting incremental PI algorithm is executed on the ESP32 input/output interface. Figure 7.2 shows the corresponding arrangement for the motor rig.

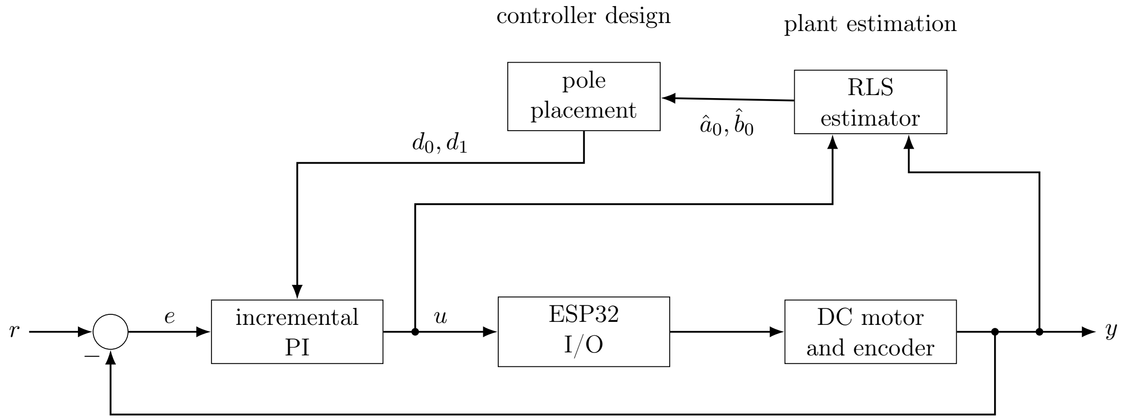

Figure 7.2: Self-tuning regulator used for the ESP32 DC motor rig.

Example 7.7: Numerical constants for the ESP32 motor rig

The sampled controller uses the period $T = 0.08\,\mathrm{s}$. The estimator is initialised with $P(0) = 100I$ and $x_e(0) = \begin{bmatrix}-0.5 & 1.0\end{bmatrix}^T$. The forgetting factor used for the validated run is $\alpha = 0.98$, the desired polynomial is $q(z) = z^2 - 0.5z + 0.06$, and the motor command is saturated at ± 255. On the hardware, the PWM output is GPIO 14, the two H-bridge direction inputs are GPIO 26 and GPIO 27, and the quadrature encoder signals are read on GPIO 32 and GPIO 33.

7.5.1 Plant model

The motor is modelled by the first-order armature-to-speed relation used in the reader. Electrical dynamics are neglected, so that the inertia J, viscous friction coefficient b, motor constant K_m, input u, and angular speed ω satisfy

$$J\dot{\omega} = -b\omega + K_m u. \tag{7.17}$$

The corresponding continuous-time transfer function is

$$G(s) = \frac{\omega(s)}{u(s)} = \frac{K_m/J}{s + b/J}. \tag{7.18}$$

With zero-order hold sampling and sampling period $T = 0.08\,\mathrm{s}$, the discrete transfer function is written in the notation of the reader as

$$G(z) = \frac{b_0}{z + a_0}, \tag{7.19}$$

where

$$a_0 = -\exp\left(-\frac{bT}{J}\right), \qquad b_0 = \frac{K_m}{b}\left(1 - \exp\left(-\frac{bT}{J}\right)\right). \tag{7.20}$$

The identified parameter vector is therefore

$$x_e = \begin{bmatrix} a_0 & b_0 \end{bmatrix}^T. \tag{7.21}$$

7.5.2 Recursive least squares

The discrete plant model in (7.19) gives the one-step regression

$$y(k) = C(k)x_e(k), \qquad C(k) = \begin{bmatrix} -y(k-1) & u(k-1) \end{bmatrix}. \tag{7.22}$$

The recursive least-squares update with forgetting factor α is

$$L(k) = \frac{P(k-1)C^T(k)}{C(k)P(k-1)C^T(k) + \alpha}, \tag{7.23}$$

$$x_e(k) = x_e(k-1) + L(k)\left(y(k) - C(k)x_e(k-1)\right), \tag{7.24}$$

$$P(k) = \frac{\left(I - L(k)C(k)\right)P(k-1)}{\alpha}. \tag{7.25}$$

The initial values are

$$P(0) = 100I, \qquad x_e(0) = \begin{bmatrix} -0.5 & 1.0 \end{bmatrix}^T. \tag{7.26}$$

Two implementation details are added to the textbook recursion. First, the covariance update is suspended when the input/output data do not contain excitation. Without this excitation gate, the division by $\alpha < 1$ in (7.25) increases P without bound while the motor is at rest. This covariance windup makes the subsequent parameter transient too aggressive and can lead to bursting. Secondly, the controller-design step uses a guard $|b_0| > 10^{-3}$. If the estimated numerator approaches zero, the pole-placement formula would otherwise divide by a nearly singular plant gain.

7.5.3 Pole placement and PI control

The desired closed-loop polynomial is chosen as

$$q(z) = z^2 - 0.5z + 0.06 = (z - 0.3)(z - 0.2). \tag{7.27}$$

For $T = 0.08\,\mathrm{s}$, these discrete poles correspond to continuous poles close to $-15\,\mathrm{s}^{-1}$ and $-20\,\mathrm{s}^{-1}$. The target settling time is therefore about 250 ms.

Writing $q(z) = z^2 + q_1 z + q_0$, the controller parameters are computed from the current plant estimates by

$$d_0 = \frac{q_0 + a_0}{b_0}, \qquad d_1 = \frac{q_1 + 1 - a_0}{b_0}. \tag{7.28}$$

The implemented controller is the incremental PI law

$$u(k) = u(k-1) + d_1 e(k) + d_0 e(k-1), \tag{7.29}$$

where $e(k) = r(k) - y(k)$. The command is saturated at ± 255. The saturated value, not the unsaturated internal value, is fed back as $u(k-1)$ in (7.29); this is the anti-windup mechanism used in the implementation.

The transient cannot be shaped by the pole locations alone. The closed-loop transfer function contains the controller zero

$$z_0 = -\frac{d_0}{d_1}. \tag{7.30}$$

This is the same structural effect described in the reader as a phase error caused by the different numerator order. A one-degree-of-freedom controller does not provide an independent numerator design, and therefore cannot remove this zero without changing the controller structure.

7.5.4 Simulation

The validation run uses the normalised plant $K_m = 1$, $b = 1$. The inertia switches from the small flywheel to the large flywheel and back to the small flywheel, abbreviated as S–L–S, with

$$J = 1 \qquad \text{and} \qquad J = 24. \tag{7.31}$$

The ratio 24:1 corresponds to the two flywheels. The run lasts 96 s. The reference is a rectangular speed command with amplitude 50 rad/s and period 16 s, starting at $t = 4$ s from standstill. Measurement noise is included.

Figure 7.3 shows the validated run for $\alpha = 0.98$. Before the inertia change, the overshoot lies between 0.74 % and 0.84 %. The 2% settling time is 0.40 s. After the L–S change the first transient in the figure reaches 1.83 % overshoot and settles in 0.56 s. The next two transients give 0.78 % with 0.48 s, and 0.77 % with 0.40 s, respectively.

The motor command remains inside the saturation limits ± 255. During standstill the trace of P remains constant at 200, showing that the excitation gate prevents covariance windup. The maximum trace over the whole run is also 200. In Figure 7.3, after the S–L inertia change, b_0 reconverges after 12.24 s, or 8.24 s after the first exciting reference edge. After the L–S change, the command sign changes at 0.12 times per second; this does not indicate a limit cycle.

The forgetting factor is deliberately set to $\alpha = 0.98$, rather than $\alpha = 1.0$. Figure 7.4 shows the decisive case after the L–S inertia change. With $\alpha = 1.0$, the covariance matrix has already become so small at the swap that the estimator can no longer learn the inertia change by a factor of 24. The loop ends in a limit cycle with command chattering between the saturation limits, which increases mechanical loading. With $\alpha = 0.98$, the estimator remains able to learn and re-adapts. Therefore $\alpha = 0.98$ is used.

7.5.5 Second-order plant

If the electrical dynamics are not neglected, the motor is no longer a first-order plant. With armature inductance L_a, armature resistance R_a, inertia J, viscous friction b, and motor constant K_m, the continuous-time transfer function becomes

$$G(s) = \frac{K_m}{L_a J s^2 + (L_a b + R_a J)\, s + (R_a b + K_m^2)}. \tag{7.32}$$

After zero-order-hold discretisation this is written as

$$G(z) = \frac{b_1 z + b_0}{z^2 + a_1 z + a_0}. \tag{7.33}$$

The adaptive estimator therefore has four parameters instead of two. Its regression vector is

$$\varphi(k) = \begin{bmatrix} -y(k-1) & -y(k-2) & u(k-1) & u(k-2) \end{bmatrix}, \tag{7.34}$$

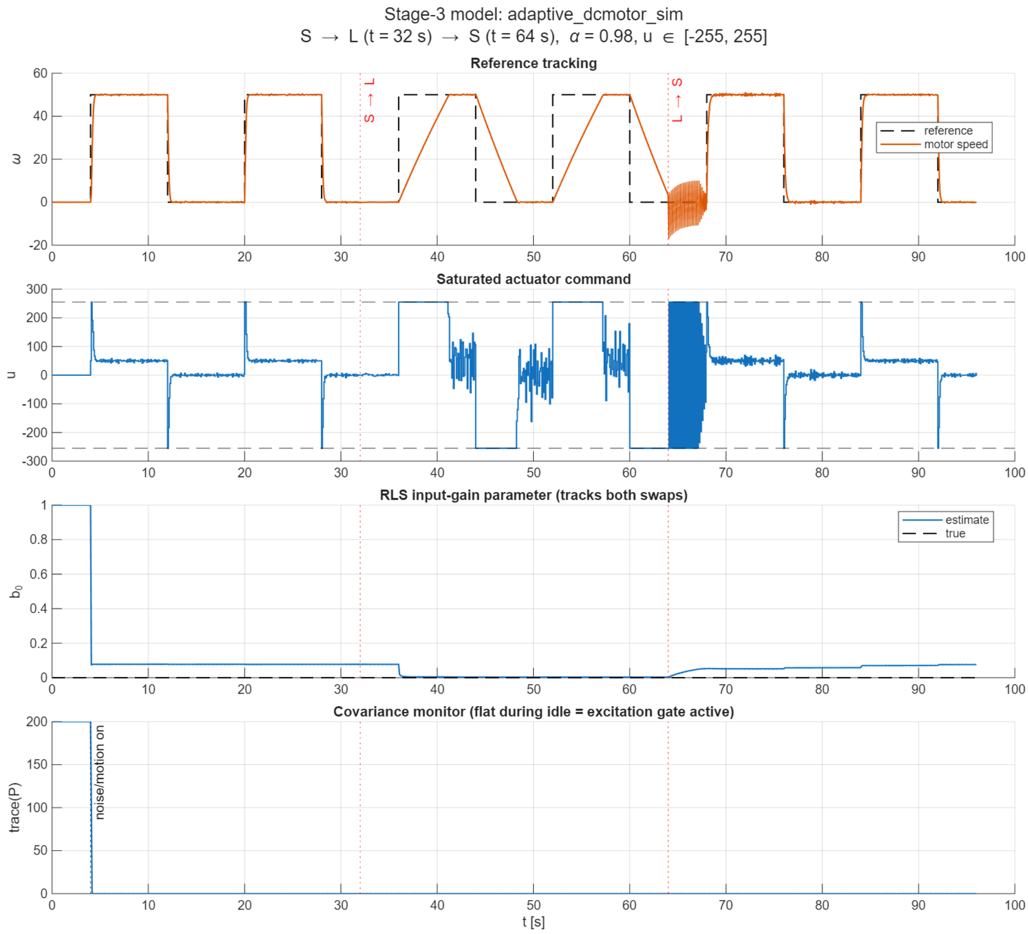

Figure 7.3: Validation run of the first-order self-tuning regulator over the S–L–S inertia change: reference tracking, saturated command, estimated b_0 against its true value, and trace(P).

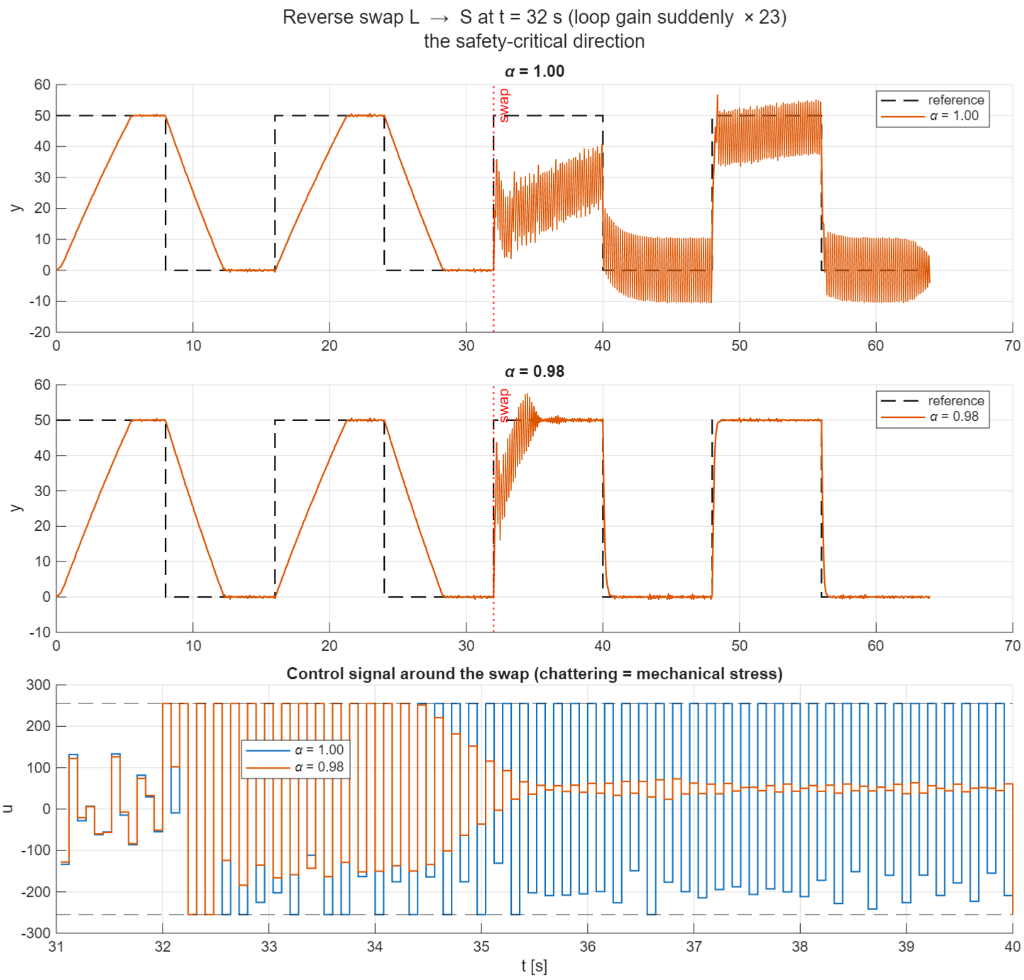

Figure 7.4: Behaviour after the L–S inertia change for $\alpha = 1.00$ (top) and $\alpha = 0.98$ (middle), with the command signal around the swap (bottom).

Table 7.1: Exact ZOH parameters of the second-order motor model.

T	a_1	a_0	b_1	b_0	z-poles	
80 ms	−0.8692	0.000000	1.8467	0.0218	0.8692	0
2 ms	−1.1322	0.1352	0.0284	0.0148	0.9965	0.1357

and the controller is no longer the incremental PI law of (7.29). It becomes an RST controller whose parameters are computed at each sampling instant from the Diophantine system given in equation 7.16 of the reader.

The decisive point is not the algebraic increase from two to four parameters, but the sampling time. For

$$K_m = 0.05, \quad J = 0.001\,\mathrm{kg\,m}^2, \quad b = 0.0005, \quad L_a = 2\,\mathrm{mH}, \quad R_a = 2\,\Omega,$$

the continuous poles are $-998.7\,\mathrm{s}^{-1}$ for the electrical mode and $-1.75\,\mathrm{s}^{-1}$ for the mechanical mode. The electrical time constant is therefore about 1 ms. Table 7.1 shows the exact ZOH discretisation for the slow sampling time used on the hardware and for a sampling time that resolves the electrical pole.

At $T = 80\,\mathrm{ms}$, the electrical pole is mapped to $z = 0$. It has fully decayed between two samples. Two of the four parameters then no longer contribute independent information to the measured output, the regressor loses rank, and the estimate drifts along the non-identifiable direction. The model is not merely over-parameterised at this sampling time; it is structurally not identifiable. Figure 7.5 shows this loss of information by plotting the magnitude of the smaller discrete pole against the sampling time.

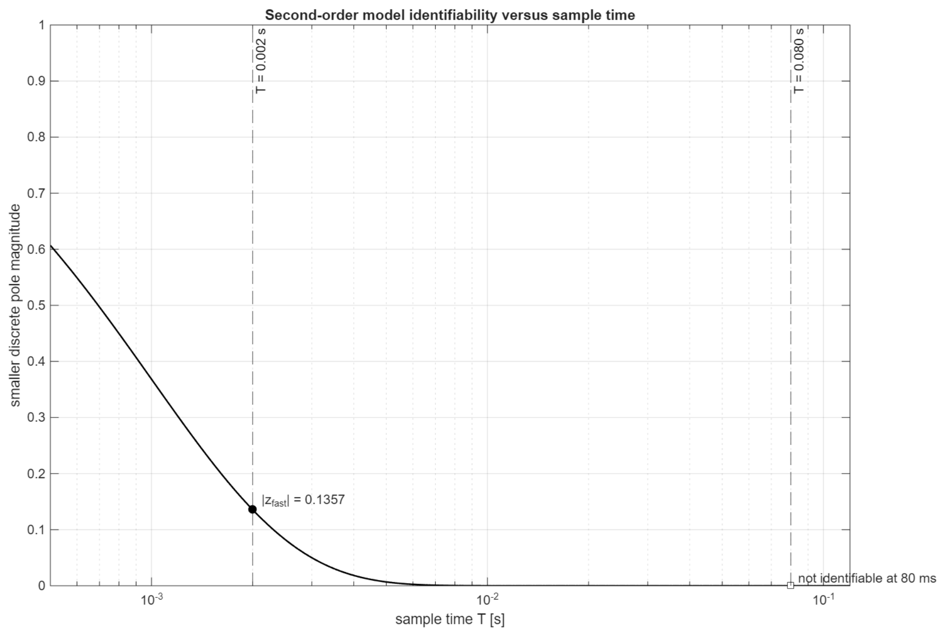

Figure 7.5: Magnitude of the smaller ZOH pole of the second-order motor model as a function of the sampling time.

The consequence is easy to overlook in practice. At $T = 80\,\mathrm{ms}$ the closed loop can still follow the reference stably, even though the parameter estimate cannot converge under that sampling. A stable closed loop is therefore not evidence of successful identification. The RST controller only needs certain combinations of the parameters, not their individual values.

Table 7.2: Validated second-order parameter estimates at $T = 2\,\mathrm{ms}$.

Parameter	Estimated, $t = 3\,\mathrm{s}$ to $4\,\mathrm{s}$	True	Absolute error
a_1	-1.131310	-1.132176	8.66×10^{-4}
a_0	0.134337	0.135200	8.63×10^{-4}
b_1	0.028348	0.028362	1.49×10^{-5}
b_0	0.014889	0.014832	5.71×10^{-5}

The working design uses $T = 2\,\mathrm{ms}$. Persistent excitation is supplied by adding a PRBS dither to the manipulated variable. The dither is not added to the reference: a controller that is asked to track a fast rectangular reference drives the actuator into saturation, and an identification experiment with a saturating actuator is worthless. Four parameters require excitation of the corresponding order.

The estimator is used without forgetting, $\alpha = 1$. Without persistent excitation, $\alpha < 1$ makes P grow without bound in the unexcited directions. This covariance windup makes the estimate drift and can destabilise the controller. In simulation, the control error grew to 88 %.

Integral action is also required. Pure pole placement fixes only the poles, not the static gain, and therefore leaves a steady-state error. The simulated value was 7.8 %. The controller denominator is therefore chosen as

$$c(z) = (z - 1)(z + c_0), \tag{7.35}$$

which gives $T(1) = 1$.

The desired closed-loop polynomial is

$$q(z) = z(z - 0.1357)(z - 0.9608)^2. \tag{7.36}$$

The fast electrical pole is deliberately left in place. If instead

$$q(z) = (z - 0.9608)^4$$

is demanded, the design slows the electrical pole by a factor of 50. The controller must then invert the fast plant dynamics and becomes unstable itself; the controller pole is at $|z| = 2.66$. Behind a saturation limit, such a controller necessarily diverges. The design rule is therefore to leave modes that are much faster than the desired bandwidth where they are.

The validation run with $T = 2\,\mathrm{ms}$ is summarised in Figure 7.6. All four parameters converge to their true ZOH values. In the window $t = 3\,\mathrm{s}$ to $4\,\mathrm{s}$, all four parameters lie within 0.65 % relative error of their true ZOH values.

The static closed-loop gain is $T(1) = 1.000$. For the step at $t = 5.5\,\mathrm{s}$, the response has no overshoot and the 2% settling time is 0.384 s. The remaining steady-state errors in the settled windows are 0.000 %, 0.000 %, and 0.002 %. The trace of P remains bounded at 400.

The first-order controller needs only two parameters, coarse excitation, and $T = 80\,\mathrm{ms}$, and it runs on the hardware. The second-order controller describes the plant more accurately, but it requires a sampling time that is 40 times shorter, targeted excitation, and much more care in the forgetting factor, integral action, and pole choice. This extra effort is worthwhile only when the electrical dynamics are relevant in the frequency range of interest.

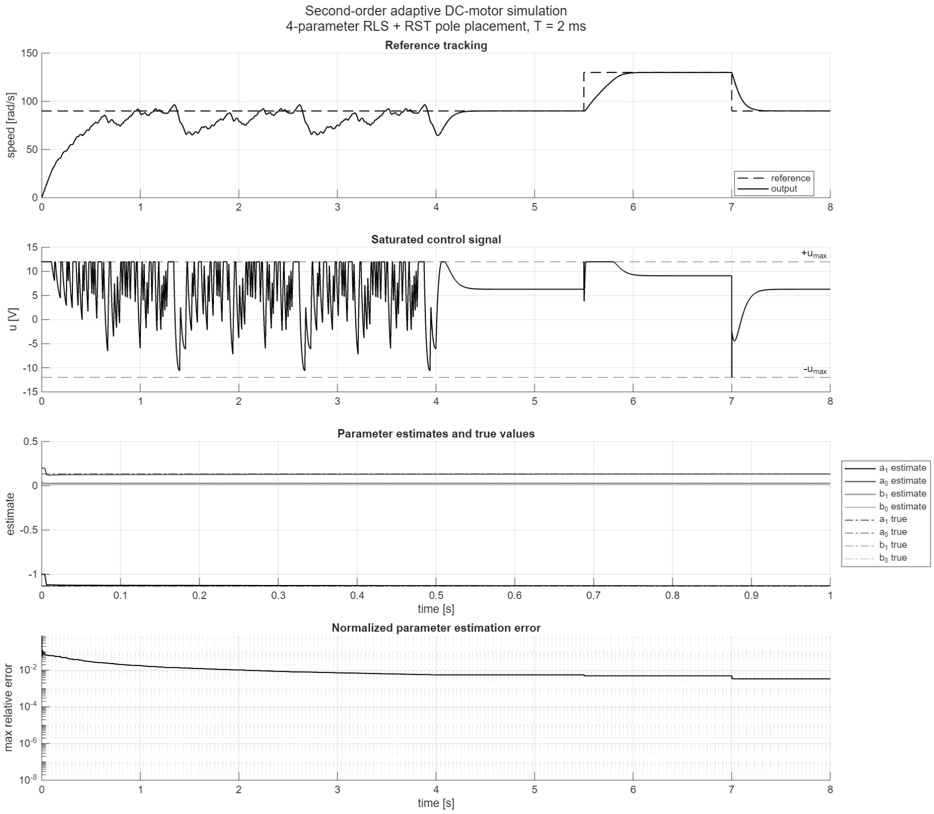

Figure 7.6: Validation run of the second-order adaptive RST controller with $T = 2\,\mathrm{ms}$, persistent excitation, integral action, and the electrical pole retained in the desired polynomial.

Table 7.3: ESP32 pin assignment for the DC motor test rig.

Signal	ESP32 pin
PWM	GPIO 14
IN1	GPIO 26
IN2	GPIO 27
Encoder A	GPIO 32
Encoder B	GPIO 33

7.5.6 Hardware realisation

The hardware platform is an ESP32-WROOM module connected to an L298N H-bridge. The L298N introduces an approximate voltage drop of 4 V, so with a 12 V supply the motor sees about 8 V. The motor is equipped with a quadrature encoder. The sampling time is $T = 0.08\,\mathrm{s}$.

The controller is implemented by automatic code generation from Simulink using the Embedded Real-Time (ERT) target and the ESP32 target support. External Mode is used for Monitor and Tune operation during experiments.

The encoder resolution was determined by a one-revolution test as 44 counts per revolution, corresponding to 11 encoder impulses with x4 quadrature edge decoding. The speed conversion

$$\omega = \frac{2\pi \Delta N}{T\,\mathrm{CPR}}$$

is therefore calibrated for CPR = 44, and the hardware speed values are reported in physical units. The measured full-duty speed of approximately 780 rad/s corresponds to about 7450 rpm. This is consistent with the gearbox-free 25GA371 speed range of roughly 3000 rpm to 8000 rpm and independently supports the encoder scaling. Hardware settling times are not stated here.

7.5.7 Conclusion

The first-order self-tuning regulator from the reader was transferred to an ESP32-based DC motor rig. In simulation, the excitation gate and the $|b_0| > 10^{-3}$ guard keep the indirect adaptive loop well behaved during standstill and inertia changes. The validated run shows bounded command effort, fast reconvergence of the plant numerator after the S–L change, and no limit-cycle behaviour after the L–S change when $\alpha = 0.98$ is used.

The hardware implementation is operational, and the encoder scaling is fixed by the one-revolution test at 44 counts per revolution. The measured speeds are therefore on a physical scale, not in arbitrary encoder counts. No quantitative hardware step responses were recorded, so no hardware settling times or overshoot values are reported; this is a limitation of the recorded measurement data, not of the encoder calibration. The inertia change was demonstrated only in simulation, because the flywheels were not mounted for the submission. On the hardware, only the first-order controller is running. The second-order design remains a simulation result, because it needs $T = 2$ ms, whereas at the hardware sampling time of $T = 80$ ms the second-order plant is structurally not identifiable.

Bibliography

[1] A. Ravazzolo-Mehrle, *Advanced Control Engineering 2 - Optimal Control.* MCI Innsbruck, 2026. Reader, spring term 2026.

[2] Abram and Fieg, "Adaptive control experiment with an arduino due." cited in [Reader].

[3] K. J. Aström and B. Wittenmark, *Adaptive Control.* Addison-Wesley, 2 ed., 1995.

[4] L. Ljung, *System Identification - Theory for the User.* Prentice Hall, 2 ed., 1999.